



Projet n°7 : “Implémentez un modèle de scoring”

Soutenance de Projet
2025-05-29
Gaillot Dimitri

SOMMAIRE

01

CONTEXTE

Rappel de la problématique,
jeu de données

02

MODELE

Préparation des données,
Modélisation, MLFlow, Data Drift

03

API

Prédictions avec modèle,
Points d'appel, Tests

04

DASHBOARD

Appels API,
Mise en forme & export

05

DEPLOIEMENT

Docker : local & production sur
Google Cloud

01

CONTEXTE



Crédit “Prêt à dépenser” Objectifs



- 1 Mettre en place un **modèle de “scoring crédit”** pour calculer la **probabilité** qu’un client rembourse son crédit. Classifier la demande en **crédit accordé / refusé**.
- 2 Développer un **dashboard interactif** permettant aux chargés de relation client d’interpréter les prédictions et d’améliorer leur connaissance client
- 3 Mettre en **production** le modèle de scoring à l’aide d’une **API** et du **dashboard** interactif utilisant l’API pour les prédictions.

Jeux de données

application_{train|test}.csv

Table **principale**, séparée en deux :
Train (avec TARGET) et **Test** (sans TARGET).
Une ligne = un prêt.

bureau.csv

Crédits passés fournis par d'autres institutions financières.
Une ligne pour chaque crédit extérieur existant avant la demande.

bureau_balance.csv

Soldes mensuels des crédits passés liés au Bureau.
Une ligne par mois d'historique de chaque crédit dans le Bureau.

POS_CASH_balance.csv

Instantanés des **soldes mensuels** des **crédits POS** (point de vente) & **crédits cash** avec Home Credit.
Une ligne pour chaque mois d'historique de crédits antérieurs.

credit_card_balance.csv

Instantanés des **soldes mensuels** des **soldes de cartes de crédit** que le demandeur possède chez Home Credit.
Une ligne pour chaque mois d'historique de crédits antérieurs.

Jeux de données

previous_application.csv

Intégralité des **demandes passées de crédits** chez Home Credit pour les demandeurs concernés.
Une ligne pour chaque mois de crédit passé.

installments_payments.csv

Historique de **remboursement de crédits** de demandeurs concernés chez Home Credit.
Une ligne pour chaque paiement effectué ou manqué.

**HomeCredit_columns
_description.csv**

Descriptions des colonnes dans les différents fichiers.



02

Modèle

Le **nettoyage** et **l'agrégation** des données ont été accélérés par l'usage d'un extrait de **Kernel Kaggle**.

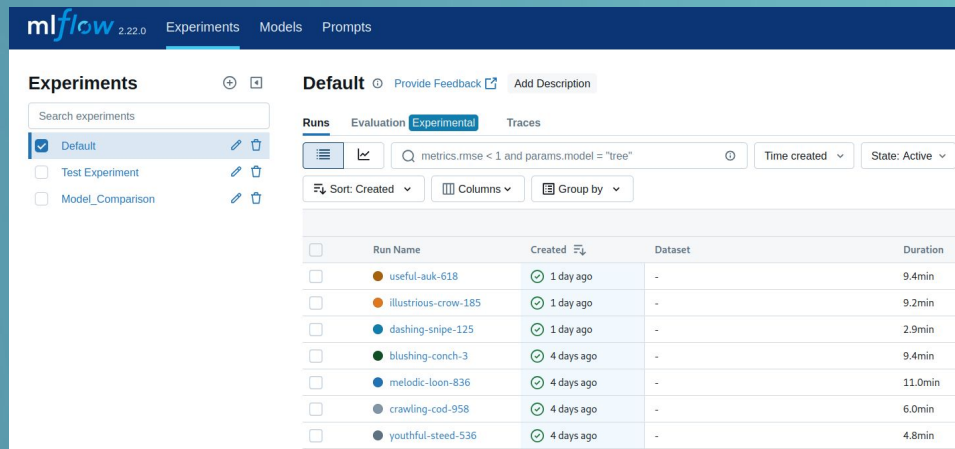
Dans un soucis de rapidité d'exécution, sur les 798 features du Kernel concerné **j'ai conservé 36 features** parmi celles ayant le plus d'importance lors d'un premier jet de modélisation sur 3 modèles :

- **XGBoost** : boosting utilisant des arbres de décision, optimisé pour la performance et l'efficacité.
- **LGBM** : boosting basé sur des arbres de décision, conçu pour être rapide et efficace en termes de mémoire, avec une approche de croissance des feuilles.
- **RandomForest** : ensembliste utilisant plusieurs arbres de décision, chacun entraîné sur un sous-ensemble de données, pour améliorer la généralisation et réduire le surapprentissage.



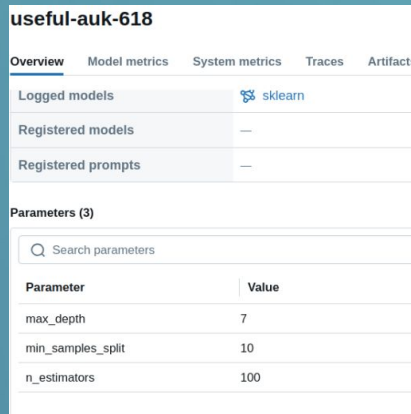
Caractéristiques principales :

- **MLFlow** pour suivi des résultats (logs entraînements, métriques)
- **F-beta score personnalisé** pour intégrer la notion de “**Score métier**”
- **Smote** optionnel pour gérer la distribution inégale des cibles (90-10)
- **GridSearchCV** pour recherche d'Hyperparamètres
- Export **Pickle** (.pkl) des meilleurs modèles pour usage ultérieur



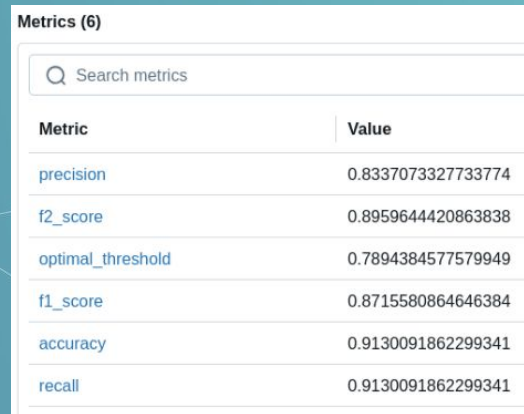
The screenshot shows the MLFlow Experiments page. On the left, there's a sidebar with 'Experiments' and a list of experiments: 'Default' (selected), 'Test Experiment', and 'Model Comparison'. The main area shows the 'Default' experiment details. It includes a search bar with the query 'metrics.rmse < 1 and params.model = "tree"', filters for 'Time created' and 'State: Active', and a table of runs.

Run Name	Created	Dataset	Duration
useful-auk-618	1 day ago	-	9.4min
illustrious-crow-185	1 day ago	-	9.2min
dashing-snipe-125	1 day ago	-	2.9min
blushing-conch-3	4 days ago	-	9.4min
melodic-loon-836	4 days ago	-	11.0min
crawling-cod-958	4 days ago	-	6.0min
youthful-steed-536	4 days ago	-	4.8min



The screenshot shows the details for the 'useful-auk-618' experiment. It has tabs for 'Overview', 'Model metrics', 'System metrics', 'Traces', and 'Artifacts'. The 'Overview' tab is active, showing 'Logged models' (sklearn), 'Registered models' (—), and 'Registered prompts' (—). Below, the 'Parameters (3)' section lists three parameters: 'max_depth' (7), 'min_samples_split' (10), and 'n_estimators' (100).

Parameter	Value
max_depth	7
min_samples_split	10
n_estimators	100



The screenshot shows the 'Metrics (6)' section for the 'useful-auk-618' experiment. It displays a table with six metrics and their values.

Metric	Value
precision	0.8337073327733774
f2_score	0.8959644420863838
optimal_threshold	0.7894384577579949
f1_score	0.8715580864646384
accuracy	0.9130091862299341
recall	0.9130091862299341

Modèle

Outil : librairie **Evidently**.

Comparaison de deux Datasets :

- dataset **d'entraînement**
- **second** dataset fourni avec des **données plus récentes**

Résultats :

- La majorité des variables n'ont pas de drift/dérive de données
- 1 variable avec un drift mineur (dépasse à peine la limite)
- PAYMENT_RATE avec un drift important

=> PAYMENT_RATE fait partie des 5 variables ayant le plus de poids, il faudra ré-entraîner le modèle.

Data Drift

Dataset Drift		
Dataset Drift is NOT detected. Dataset drift detection threshold is 0.5		
36 Columns	2 Drifted Columns	0.0556 Share of Drifted Columns
Data Drift Summary		
Drift is detected for 5.556% of columns (2 out of 36).		

New version 🌟 ⓘ				Metrics	Tests
35 SUCCESS	0 WARNING	2 FAIL	0 ERROR		
By test status ▾					
Failed tests					Hide
❗ Value Drift for column PAYMENT_RATE					
Drift score is 1.60. The drift detection method is PSI. The drift threshold is 0.10.					
❗ Value Drift for column AMT_GOODS_PRICE					
Drift score is 0.11. The drift detection method is PSI. The drift threshold is 0.10.					



03

API

Librairie FastAPI.

- Utilise le modèle exporté
- 6 points d'entrée :
 - **/data** : retourne l'**ensemble des données utilisateur** (format Parquet ou CSV).
 - **/user_data** : demande un ID client, retourne les **données du client**.
 - **/shap** : demande un ID client, retourne les données **SHAP** le concernant par rapport au modèle.
 - **/shap_by_input** : demande le détail des données client, retourne le **SHAP**.
 - **/predict** : demande le détail des données client, retourne une **prédiction**, les **probabilités** de la prédiction, la **prédiction binaire** correspondant au score métier et le **seuil optimal** du score métier.



Librairie PyTest

- Emule un client de test pour FastAPI
- Récupère la définition main.py de l'API
- Teste les différents points d'appel :
 - Appels corrects
 - Appels erronés
 - Structure des résultats

Les tests sont exécutables manuellement, et sont exécutés lors des déploiements (local et en production).

```
def test_get_shap_not_found():
    response = client.post("/shap", json={"SK_ID_CURR": 99999})
    assert response.status_code == 200
    assert response.json() == {"error": "404: User not found"}

def test_get_shap():
    response = client.post("/shap", json={"SK_ID_CURR": VALID_SK_ID_CURR})
    assert response.status_code == 200
    assert isinstance(response.json(), list)
    assert len(response.json()) > 0
    for item in response.json():
        assert isinstance(item, dict)
        for key, value in item.items():
            assert isinstance(value, (int, float))

def test_get_shap_not_found():
    response = client.post("/shap", json={"SK_ID_CURR": 99999})
    assert response.status_code == 200
    assert response.json() == {"error": "404: User not found"}

# Tests pour l'endpoint /shap_by_input
def test_get_shap_by_input():
    response = client.post("/shap_by_input", json=valid_query_data)
    assert response.status_code == 200
    assert isinstance(response.json(), list)
    assert len(response.json()) > 0
    for item in response.json():
        assert isinstance(item, dict)
        for key, value in item.items():
            assert isinstance(key, str)
            assert isinstance(value, (int, float))
```



04

Dashboard

Spécifications

- 1 Permettre de **visualiser le score et l'interprétation de ce score** pour chaque client de façon intelligible pour une personne non experte en data science
- 2 Permettre de **visualiser des informations descriptives** relatives à un client (via un système de filtre).
- 3 Permettre de **comparer** les informations descriptives **relatives à un client à l'ensemble des clients** ou à un groupe de clients similaires.

Dashboard

Librairie Streamlit

- **Sélection** de l'ID client
- Possibilité d'**éditer les données** pour tester l'impact sur prédiction
- **Bouton prédiction** : affiche une jauge, la prédiction, probabilité, prédiction binaire et seuil
- **Importance** des variables : **SHAP** du client
- **Positionnement** du client par variable : différent modes de sélection, sélection top 4 du SHAP par défaut, **édition des sélections possible**

Streamlit

Dashboard Client

Sélectionnez un ID client

100002

Afficher les champs modifiables

Lancer la prédiction

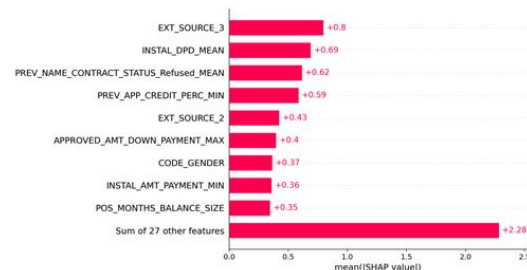
Taux d'acceptation



Décision initiale du modèle : **crédit refusé** (probabilité : 63.3%)

Décision finale de crédit : **crédit refusé** (seuil optimal : 97.95%)

Importance des variables client (SHAP)



Afficher l'intégralité du SHAP

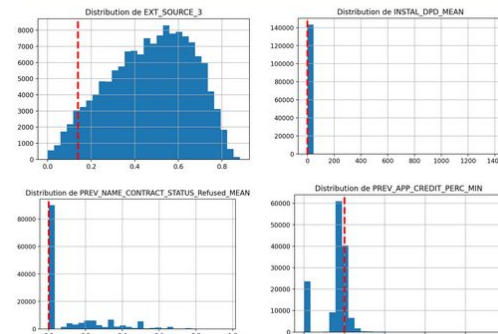
Positionnement du client par variable

☒ Mode manuel

Sélectionnez les variables à afficher

EXT_SOURCE_3 x INSTAL_DPD_MEAN x PREV_NAME_CO... x

PREV_APP_CRE... x



05

Déploiement

Déploiement

Docker + local

- **Structure** : un dossier/repo Git principal, deux sous-dossiers (API, dashboard)
- **Docker** : chaque sous-dossier a une définition Dockerfile indépendante
- **Déploiement local** : un docker-compose.yml incluant l'exécution de tests

```
projet_7 > app_oc_p7 > app_streamlit > Dockerfile
1 # Utiliser une image de base appropriée
2 FROM python:3.9
3
4 # Définir le répertoire de travail
5 WORKDIR /app
6
7 # Copier les fichiers nécessaires
8 COPY requirements.txt .
9 COPY app.py .
10
11 # Installer les dépendances
12 RUN pip install --no-cache-dir -r requirements.txt
13
14 # Exposer le port 8080
15 EXPOSE 8080
16
17 # Lancer l'application Streamlit
18 CMD ["streamlit", "run", "app.py", "--server.port=8080", "--server.address=0.0.0.0"]
```

```
projet_7 > app_oc_p7 > docker-compose.yml
1 services:
2   api:
3     build: ./app_API
4     ports:
5       - "8000:8000"
6     environment:
7       - PORT=8000
8     networks:
9       - my_network
10
11   streamlit-app:
12     build: ./app_streamlit
13     ports:
14       - "8080:8080"
15     environment:
16       - API_URL=http://api:8000/
17     depends_on:
18       - api
19     networks:
20       - my_network
21
22   test:
23     build: ./app_API
24     command: pytest -v tests/
25     volumes:
26       - ./app_API:/app
27       - ./app_API/app/model.pkl:/app/model.pkl
28     networks:
29       - my_network
30
31   networks:
32     my_network:
33       driver: bridge
34
```

```

  app_oc_p7
  .github / workflows
    .pytest_cache
    app / data
  app_API
    .pytest_cache
    app
    data
    tests
    Dockerfile
    Makefile
    model.pkl
    README.MD
  app_streamlit
    .streamlit
    .env
    app.py
    Dockerfile
    Makefile
    README.md
    requirements.txt
    data
    k8s
    docker-compose.yml
    Makefile
    README.md
```

Déploiement

- **Structure** : 2 sous-dossiers pour API & Dashboard, comme en local
- **CI/CD** Github Actions (orchestration Intégration Continue / Déploiement Continu) :
 - définition dans un fichier **deploy.yaml**
 - Variables "**Secret**" définies dans les options du repo GitHub
 - Exécution des **tests** API intégrée au déploiement
- **Google Container Registry** : build & push images Docker
- **Google Cloud Run** : hébergement

En production

CI/CD Pipeline
deploy.yaml

59 workflow runs

- ✓ update absolute data path for deploy tests
CI/CD Pipeline #59: Commit 238b15a pushed by mijkami
- ✗ revert api dockerfile data
CI/CD Pipeline #58: Commit 0685ad6 pushed by mijkami

Cloud Run Service details Edit & deploy new revision

streamlit-app Region: europe-west9 URL: <https://streamlit-app-8>

Metrics SLOs Logs Revisions Triggers Networking Sec

Revisions Manage traffic

Filter Filter revisions

Name	Traffic	Deployed ↓	Revision to
streamlit-app-00018-jmj	100% (to latest)	1 day ago	+
streamlit-app-00017-dqc	0%	1 day ago	

Setup, Build, Publish, and Deploy

succeeded yesterday in 5m 43s

- ✓ Set up job
- ✓ Checkout
- ✓ Set up Python
- ✓ Install dependencies
- ✓ Run tests
- ✓ Set up Google Cloud SDK
- ✓ Authenticate to Google Cloud
- ✓ Install gke-gcloud-auth-plugin
- ✓ Set up Docker Buildx
- ✓ Login to Google Container Registry
- ✓ Build and push API image
- ✓ Build and push Streamlit image
- ✓ Deploy API to Cloud Run
- ✓ Create .env file for Streamlit
- ✓ Deploy Streamlit App to Cloud Run
- ✓ Post Build and push Streamlit image
- ✓ Post Build and push API image
- ✓ Post Login to Google Container Registry
- ✓ Post Set up Docker Buildx
- ✓ Post Authenticate to Google Cloud
- ✓ Post Set up Python
- ✓ Post Checkout
- ✓ Complete job

Abstract geometric line art in the top corners of the slide, consisting of interconnected lines and dots forming various polygonal shapes.

Merci

Pour votre attention.

GAILLOT Dimitri